MnemosyneOS

A research microkernel for isolation and deterministic service replay



Research proposal and working prototype
October 2026

Research objective

Can a failed service recover its state without regaining revoked authority or repeating committed effects?

MnemosyneOS studies a capability-secure microkernel, deterministic service replay, and policy-checked recovery proposals.

The recovery protocol is the central research question. The custom kernel provides the experimental environment.

Why service recovery is difficult

Containment	A component failure must leave unrelated services able to run.
State	A restart needs a consistent checkpoint and an agreed input history.
Authority	Recovery must use permissions that remain valid now.
Effects	A retry can repeat an operation that another service already committed.

Research problem. The current prototype tests a deliberately restricted recovery case.

Scope and implementation status

Component	Current state	Next acceptance gate
Rust microkernel	Booting	Process lifecycle and syscall boundary
Processes and IPC	Mapping tested	Ring 3, preemption, bounded endpoints
Capabilities	Table tested	Enforce handles across processes
Service isolation	Planned	A fault leaves a sibling service running
ChronicleFS	Planned	Persistent roots and reboot recovery
Wasm runtime	Host prototype	Move runtime into a userspace service
Checkpoint / replay	Host prototype	Durable logs and guest integration
Recovery verifier	Planned	Check typed plans before execution
Fault tests / dashboard	CLI tests	Measured fault campaign and host UI

 Current implementation baseline: 44b739d. Full-system recovery remains a completion target.

Proposed system architecture

Layer	Responsibility	Authority boundary
Wasm workloads	Run bounded applications through a small import ABI	No ambient device or file access
Userspace services	IPC, ChronicleFS, checkpoints and service supervision	Explicit delegated capabilities
Recovery control	Propose, verify, stage and publish recovery actions	The proposer cannot execute directly
Rust microkernel	Address spaces, scheduling, endpoints and object handles	Kernel enforces each privileged operation

Target architecture. The implemented kernel and host runtime are currently separate.

Capability table semantics

Operation	Example	Checked outcome
Mint root	Memory object 7 with READ + DELEGATE	Trusted object manager only
Delegate	Child receives READ	Rights can only decrease
Resolve	Use READ handle to request WRITE	Denied
Revoke	Remove a derivation subtree	Descendant handles become invalid
Reuse slot	Slot receives a new generation	Old handle cannot alias the new object

Implemented table behavior. Caller-table selection and object lifecycle enforcement still need integration.

Processes, preemption and IPC

Address spaces	Enter ring 3 with separate code, data, stack and kernel entry state.
Scheduling	Preempt a spinning process on one CPU and keep a peer responsive.
Endpoints	Use bounded copy-based messages with capability checks on each call.
Fault delivery	Terminate or restart only the affected service and report a structured fault.

 Next kernel milestone. Timer interrupts currently work, but process preemption is not implemented.

The current Wasm execution profile

64 KiB

fixed linear memory

10,000

fuel per request

4,096

maximum trace events

Wasmi 2.0.0 runs one reviewed integer key-value fixture.
Its state consists of linear memory and one mutable i64 global.

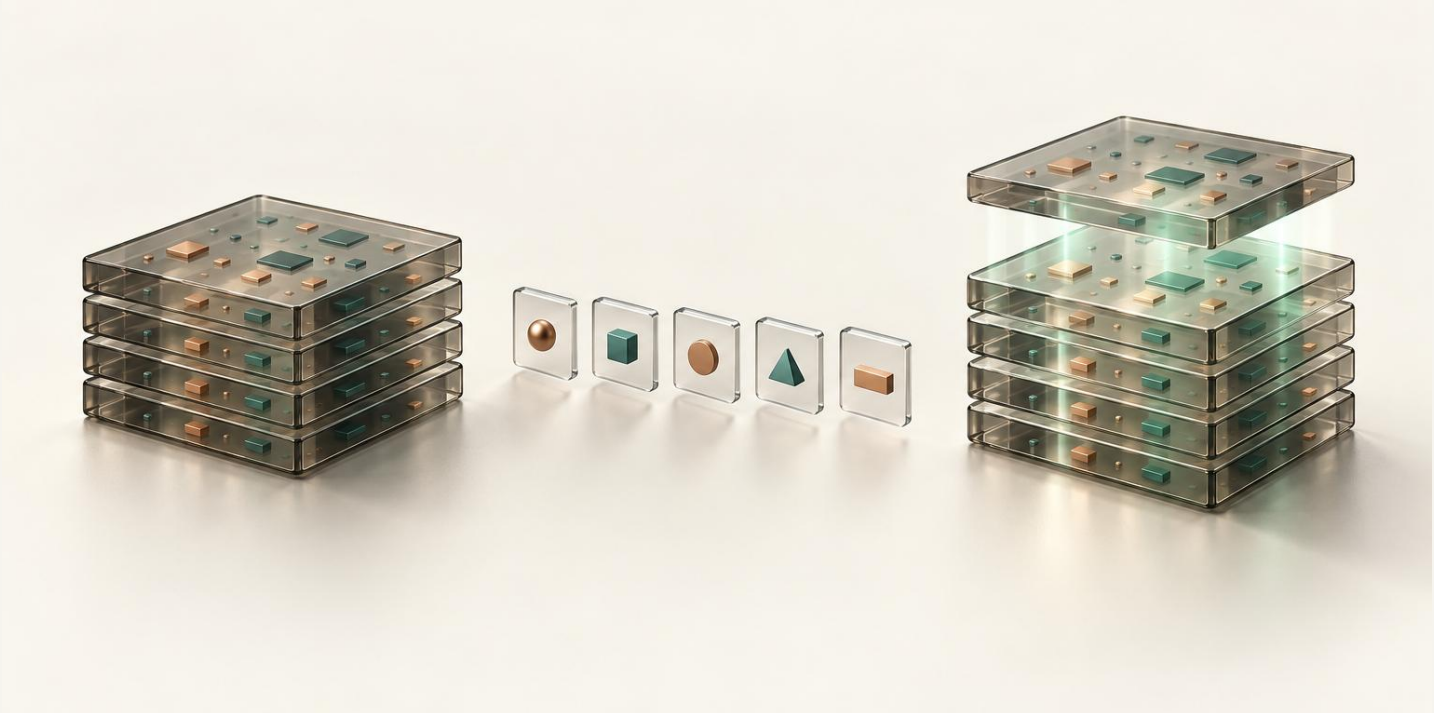
No threads, tables, floating point or start function. Host imports provide clock, random and an in-memory effect observation.

 Host prototype. These limits describe this fixture, not a universal Wasm execution guarantee.

Checkpoint and deterministic replay

A checkpoint preserves the fixture state at a completed-handler boundary.

Record the subsequent input history. Restore into a fresh instance. Inject recorded inputs and compare replies and effects.



Implemented with full state copies in the host fixture. COW and durable recovery are planned.

A strict trace contract

Event	Replay action	Failure response
Request	Compare key and delta	Reject a changed invocation
Clock	Inject the recorded value	Reject a missing or wrong event
Random	Check bound and inject value	Reject invalid bounds or results
Effect	Compare the complete payload	Discard pending effects on failure
Reply	Compare the result	Poison the instance on mismatch

Completion also requires consuming the entire log. Extra trailing events fail replay.

ChronicleFS and durable recovery

Immutable roots	A small versioned filesystem exposes snapshots through committed roots.
Commit protocol	Define write ordering, checksums and device flush assumptions explicitly.
Recovery descriptor	Bind the filesystem root, checkpoint, input position and pending effects.
Crash testing	Interrupt every persistence transition and check state after reboot.

Proposed storage design. ChronicleFS and power-loss recovery are not implemented in the baseline.

AI-assisted recovery proposals

```
plan RecoveryPlanV1
  service: kv
  checkpoint: approved_snapshot
  requires: current_capability_epoch
  limits: {steps: 4, replay_events: 4096}
  actions: [quiesce, restore, replay, validate]
  publish: only_if_authorized_and_valid
  on_failure: preserve_safe_state_and_stop
```

Proposed control flow

Structured fault trace

Untrusted plan proposal

Deterministic verification

Trusted staged execution

 Illustrative plan syntax only. The DSL, verifier, executor and AI adapter remain planned.

Deterministic verification obligations

Obligation	Required decision before execution
Capability authority	Every referenced object and operation is authorized now.
No amplification	The plan cannot mint stronger privileges during recovery.
Resource bounds	Memory, handles, log capacity and fuel fit reserved budgets.
Bounded execution	Each action terminates or reaches a defined time/resource limit.
Rollback availability	A permitted safe state remains available if a staged step fails.
State freshness	Execution rejects stale epochs and changed preconditions.

Proposed verifier specification. Parsing a typed plan alone does not establish these properties.

Working prototype evidence

24

passing host tests

3

CPU protection probes

12

requests replayed exactly

The kernel boots in debug and release configurations.
The host experiment rejects altered traces and corrupted state.

The rehearsal captures real command output and a reproducible replay report. Performance evaluation remains a later milestone.

 Recorded code baseline 44b739d. Correctness evidence applies to the stated prototype scope.

Live demo through the CLI

1 `python tools/dev.py boot`

Boot the kernel and inspect serial events

2 `python tools/dev.py test`

Check capability and replay failure cases

3 `python tools/dev.py demo`

Restore, replay and compare complete state

 Guided route: python tools/present.py. The current product interface is terminal output.

Kernel boot evidence

```
{"event":"frames_ok","count":2}
{"event":"capabilities_ok"}
{"event":"paging_ok","fault_probes":3}
{"event":"exception_ok","vector":3}
{"event":"timer_ok","ticks":5}
{"event":"boot_ok"}
```

What the events establish

Usable frames

Capability table behavior

CPU protection faults

Exception and timer delivery

 Excerpt from the saved serial log. Full log: presentation/evidence/boot.log.

Replay reaches the same fixture state

12

requests

60

events

65,544 B

mutable state

Final state SHA-256

`30097fe2565ba3716a1aa8c0699c8614caa27b9ca0edcc493a4d60b790a2da11`

Restored and replayed SHA-256

`30097fe2565ba3716a1aa8c0699c8614caa27b9ca0edcc493a4d60b790a2da11`

State equal: true Effects equal: true
Missing, reordered and extra events: rejected
Corrupted checkpoint: rejected

Actual host experiment result. Checkpoint metadata is additional to the mutable-state payload.

Reuse and research foundations

Foundation	Role in MnemosyneOS
Limine and Rust OSDev	Reused boot, CPU, UART, interrupt and paging support
Wasmi and wasm-tools	Reused interpreter, validation, fuel and WAT compilation
seL4	Reference for capability design and precise proof scope
Theseus	Reference for component state ownership and recovery
RIFL and CrashMonkey	References for effect handling and crash-test design
LLM-Modulo planning	Reference for proposer/checker separation

Dependencies are pinned and attributed. Research references are not automatically vendored code.

Evaluation plan

Measure	Controlled experiment
IPC and capability cost	Vary payload size, rights checks and table occupancy
Wasm performance	Run the same workload natively and in the interpreter
Checkpoint overhead	Measure pause time, copied bytes and write amplification
Replay fidelity	Compare state, replies and effect records after injected faults
Failure containment	Track healthy-service progress while a peer fails
Recovery time	Compare restart, checkpoint restore, replay and checked plans

Proposed experiments. Report distributions and raw artifacts. No performance claims yet.

Implementation milestones

Gate	Demonstration required to advance
1 Kernel isolation	Two ring-3 services, preemption and capability-checked IPC
2 Runtime service	The Wasm fixture runs inside an isolated guest service
3 Durable state	ChronicleFS recovers its last committed root after a crash
4 Recovery protocol	Restore and replay without duplicated committed effects
5 Checked planning	Reject forbidden plans and evaluate recovery baselines

Proposed sequence. Calendar estimates follow scope and team-capacity review.

Main engineering risks

Risk	Design response
Scope exceeds capacity	Use staged gates and a single first workload
Hidden nondeterminism	Mediate host inputs and restrict shared mutable state
Duplicate external effects	Design stable IDs, durable completion records and peer handling
Stale authorization	Bind recovery to current epochs and recheck before publication
Overstated assurance	Keep tests, policy checking and formal proofs separate

 The full protocol must address these risks before claiming safe autonomous recovery.

The project deliverable

A reproducible testbed for capability-aware service recovery

Custom Rust kernel and isolated services
A specified checkpoint and replay protocol
A checked recovery executor with an untrusted proposer
A fault suite with reproducible measurements

 Current foundation: booting kernel, capability tests and exact host-fixture replay.

Technical backup: trust model

Party or component	Assumption or obligation
Wasm application	Potentially faulty or malicious within its admitted interface
AI proposer	Untrusted and unable to bypass deterministic checks
Microkernel	Trusted enforcement of memory, CPU and object authority
Verifier and executor	Trusted state checks, bounded actions and publication logic
Log and storage service	Explicit integrity and persistence assumptions
Hardware and boot chain	Part of the stated trusted environment

Proposed threat boundary. Whole-system compromise, side channels and physical attacks need separate scope.

Technical backup: replay boundaries

Safe point	Capture state only between completed handlers in the first runtime.
State inventory	Account for globals, memory, queues, handles and pending operations.
External effects	Bind publication to durable state and receiver-side duplicate handling.
Concurrency	Coordinate related service histories before claiming group recovery.

Current lab covers one fixture and in-memory observations. General native replay is outside this milestone.

Technical backup: capability edge cases

Case	Current rule
Stale slot handle	Reject mismatched generation
Generation exhaustion	Retire the slot permanently
Delegation amplification	Reject rights outside the parent subset
Full table	Fail allocation without changing existing authority
Subtree deletion	Delete descendants before dangling ancestry can arise
Object ID reuse	Trusted teardown must invalidate all referencing tables

Implemented allocation-free table. Revocation is synchronous $O(N^2)$ within a bounded capacity.

Technical backup: demo commands

```
Set-Location 'C:\Users\ludic\Desktop\mnemosyne'
python tools/present.py

# Rehearsal without pauses
python tools/present.py --auto

# Host tests and replay only
python tools/present.py --host-only

# Saved evidence, clearly labeled
python tools/present.py --recorded
```

Preparation

Warm the build cache

Use a readable terminal font

Open the demo guide

Keep the PDF as a fallback

 Exact setup, expected output and troubleshooting: presentation/demo-runbook.md.

Technical backup: likely review questions

Why a custom kernel?

The course artifact exposes the authority and recovery boundary directly.

Why Rust?

Use ownership and types while keeping unsafe hardware code small and reviewable.

Why Wasm first?

A restricted import ABI gives a tractable state and input model.

What does AI add?

Candidate recovery strategies under the same deterministic enforcement boundary.

What is the contribution?

Evaluate recovery semantics that combine current authority, history and effects.

What proves completion?

A fault-and-recovery scenario with preserved state, authority and peer progress.

 The thesis and evaluation are proposed. Existing tests establish the current prototype behavior.

Technical backup: assurance claims

Evidence or mechanism	What it establishes	Limit
Unit tests	Expected behavior for exercised cases	No exhaustive whole-system proof
CPU fault probes	Protection on the test mapping	No process-isolation demo yet
Snapshot checksum	Detection of accidental changes	No authenticity or semantic validity
Deterministic verifier	Planned rejection against a policy	Needs a specification and sound executor
Formal proof	A theorem under explicit assumptions	Not completed for MnemosyneOS

 “Verified AI-assisted recovery” names the proposed check-before-execution objective.

Formal project proposal

MnemosyneOS: Design and Implementation of a Capability-Secure Microkernel with Deterministic Service Replay and Verified AI-Assisted Recovery

MnemosyneOS is a research-oriented 64-bit microkernel designed to isolate, monitor and recover autonomous and potentially untrusted workloads. The system implements virtual memory, preemptive process scheduling, capability-based access control, inter-process communication, userspace services, a transactional snapshotting filesystem and a WebAssembly application runtime. Individual services may be checkpointed using copy-on-write memory and replayed using recorded IPC messages and nondeterministic system inputs. An AI-assisted control plane analyzes structured fault traces and proposes recovery actions in a typed policy language. Proposed actions are checked by a deterministic verifier for capability authority, resource bounds, privilege amplification, bounded execution and rollback availability before they are executed. The project evaluates IPC overhead, capability-validation cost, WebAssembly performance, checkpoint overhead, replay fidelity, failure-containment effectiveness and mean recovery time under injected faults.

Selected papers and source repositories

seL4	Klein et al., SOSP 2009
Theseus	Boos et al., OSDI 2020
RIFL	Lee et al., SOSP 2015
CrashMonkey / B3	Mohan et al., OSDI 2018
LLM-Modulo planning	Gundawar et al., arXiv 2024
Implementation reuse	Limine, Rust OSDev, Wasmi, wasm-tools

 Full citations and links appear in speaker notes. Repository research docs provide the extended review.